HACKMATRIX 2026

Link & Scam Detective

Complete implementation brief for the Antigravity coding agent

AI-Powered Web Threat Detection & Digital Trust Intelligence
Artificial Intelligence + Cybersecurity + Web3

Item	Detail
Team	APEX WARCODE (HM-6FC44F) - Somnath Pal (Leader), Sreya Roy, Sudhansu Kumar, Sukriti Singh
Hard deadline	10 November 2026, 8:00 PM IST. Target: submit on the HackMatrix dashboard by 7:00 PM IST at the latest.
Document date	9 November 2026
Document type	Specification only. It deliberately contains NO source code. Every behaviour, rule, field, endpoint, threshold and test is described in plain language so that an AI coding agent can implement it exactly.
Reading order	Sections 1 to 8 define the rules and data. Section 9 is the step-by-step build plan (Phases 0 to 10). Sections 10 to 14 cover the API, failure handling, tests, time plan and submission. Appendices hold the data lists the app needs.

This brief reflects the official HackMatrix problem statement. Where it adds detail, the added detail is a design decision made to satisfy the problem statement, and it is marked as such.

Contents

1. Purpose and how the agent must work . . . 3

2. Product definition and success criteria . . . 3

3. Hard constraints (non-negotiable) . . . 5

4. Human prerequisites (do these first, in parallel) . . . 6

5. Fixed technology decisions . . . 7

6. Repository layout . . . 7

7. Data contracts . . . 9

8. Scoring and decision rules . . . 11

9. Phased build plan . . . 14

Phase 0 - Foundations (about 1.5 hours) . . . 14

Phase 1 - SSRF guard and safe HTTP client (about 1.5 hours) . . . 14

Phase 2 - URL and domain analyser (about 3 hours) . . . 15

Phase 3 - TLS certificate and security-header analysis (about 3 hours) . . . 16

Phase 4 - Threat intelligence (about 2 hours) . . . 17

Phase 5 - Scoring engine and report assembly (about 2 hours) . . . 18

Phase 6 - Message analyser and AI explanation (about 4 hours) . . . 19

Phase 7 - Persistence and campaign correlation (about 2.5 hours) . . . 20

Phase 8 - Web3 proof registry (about 4 hours) . . . 20

Phase 9 - Frontend dashboard (about 5 hours, can overlap Phases 6 to 8) . . . 22

Phase 10 - Fixtures, tests, documentation and deployment (about 5 hours) . . . 23

10. API specification . . . 25

11. Failure handling and graceful degradation . . . 25

12. Test plan . . . 27

13. Time plan, roles and cut lines . . . 29

14. Documentation and submission . . . 29

Appendix A - Data files the app needs . . . 31

Appendix B - Keyword lists . . . 32

Appendix C - Suspicious top-level domains and URL shorteners . . . 32

Appendix D - Plain-language text library for findings . . . 32

Appendix E - Message phrase categories (starter examples) . . . 33

Appendix F - Exact user-interface copy . . . 33

Appendix G - Glossary . . . 34

Appendix H - Definition of done . . . 34

1. Purpose and how the agent must work

You are building a web application named **Link & Scam Detective** in roughly 30 working hours. A user submits a suspicious link, domain, or pasted SMS/email. The app runs safe, passive cybersecurity checks, uses an AI model to explain the findings in plain English, assigns one of five fixed risk labels, links the scan to earlier related scans, and lets the user save a tamper-evident fingerprint (hash) of the report to a blockchain testnet and later verify it.

Working agreement for the coding agent

1. Read this whole document once before creating any file. Then produce a written implementation plan that maps to Phases 0 to 10 in Section 9, and only then start building.
2. Build phase by phase in the given order. Each phase ends with acceptance checks. Run them. Do not start the next phase until the current one passes. If a check fails, fix the cause; never weaken the check.
3. The Hard Constraints in Section 3 override every convenience, shortcut and personal preference. If something here seems to conflict with a hard constraint, the hard constraint wins.
4. Prefer a smaller feature that works over a larger feature that is flaky. Judges score demonstrated functionality, not the number of integrations. Every external call must have a timeout and a graceful failure path that records a Limitation in the report instead of crashing.
5. Never invent facts. Data such as fingerprints of distrusted certificate authorities must come from a cited public source with its URL recorded. If a value cannot be verified, mark the entry as unverified and score it lower (rules in Section 8).
6. Never place a secret in the repository, the frontend bundle, logs or screenshots. Provide an example environment file that lists variable names only.
7. Commit after every phase with a clear message. Keep the README updated as you go, not at the end.
8. If your environment gives you a terminal and a browser, use them: run the automated tests after each phase, and at the end click through every demo button in the browser and confirm the visible results match Section 12.
9. Ask the human only for the items in Section 4 (accounts, keys, funded wallet). Make every other decision yourself using this document.

> **AGENT INSTRUCTION**
> Suggested first message to give the agent: "Read the attached implementation brief completely. Create an implementation plan for Phases 0 to 10, list any human prerequisites you are blocked on, then start Phase 0. After every phase run its acceptance checks and report the results before continuing. Obey the Hard Constraints in Section 3 at all times."

2. Product definition and success criteria

The four core jobs

Job	What it must do	Weight
Cybersecurity scanner	Detect lookalike and impersonation domains, inspect the TLS certificate and its chain, inspect security headers, follow redirects safely, and query at least one threat-intelligence source.	30%
AI detective	Analyse pasted SMS/email for social-engineering patterns, and write evidence-grounded plain-English explanations for both URL and message scans. The AI explains; it never decides the label.	20%
Campaign linker	Deterministically match a new scan against earlier scans by shared IP, domain, certificate fingerprint or message template, and show a "Linked to Existing Phishing Campaign" badge.	part of AI + Cyber
Web3 proof registry	Hash the full report, store the report off-chain, store only hash, timestamp, status, reporter and revision on a smart contract, and provide a Verify Integrity button.	20%

The remaining 30% is Technical Implementation (20%: working architecture and integration) and Innovation / User Experience (10%). The brief states that judges evaluate demonstrated functionality, so a smooth, reliable end-to-end demo matters more than breadth.

The five-step user workflow the UI must support

1. Submit: the user pastes a link or domain, or pastes a message/email, or clicks one of six demo buttons.

2. Investigate: the platform collects domain information, redirects, certificate details and threat intelligence.
3. Analyse: rule-based checks flag suspicious patterns; AI correlates and explains in plain language.
4. Assess risk: a structured report shows label, evidence, sources, timestamp, coverage, actions and limitations.
5. Report and verify: the user saves the report; its hash is committed on-chain; anyone can verify integrity later.

The five risk labels (exact wording, no others)

Label	Meaning (from the brief)	When the system may use it
Low Risk	No significant malicious indicators identified within the available checks	Only when little or no risk evidence AND evidence coverage is adequate (Section 8)
Suspicious	One or more indicators require further investigation	Weak or single-category evidence; any impersonation signal
High Risk	Multiple significant indicators suggest a substantial security threat	Strong score across at least two independent categories
Confirmed Malicious	Supported by authoritative threat intelligence or sufficiently validated evidence	Only on an authoritative feed match defined in Section 8
Inconclusive	Insufficient data or unavailable verification sources	Low coverage or unreachable target without strong signals

Every report must contain

- Overall risk label; identified threat indicators; supporting technical evidence; the source of each important finding.
- Analysis timestamp (UTC); evidence coverage (a percentage and which checks ran); recommended actions.
- Limitations and a list of checks that were unavailable.
- A visible note that no automated tool can guarantee a website is completely safe.

Definition of a successful demo (about three minutes)

1. Click a safe sample: a Low Risk report appears with worded evidence coverage and the standard disclaimer.
2. Click a lookalike bank URL: a High Risk or Suspicious report appears with brand-impersonation evidence, a valid-HTTPS warning, a plain-English AI explanation, and (if seeded history exists) the campaign badge.
3. Paste the scam SMS: extracted links and phone numbers appear, urgency and credential-request signals are listed, and the AI explanation cites only listed evidence.
4. Type a localhost or cloud-metadata address: the SSRF guard visibly blocks it.
5. Click Save and Verify: a transaction hash and explorer link appear. Click Verify Integrity: result is MATCH.
6. Use the demo-mode tamper button, verify again: result is MISMATCH. Restore and verify: MATCH again.

3. Hard constraints (non-negotiable)

Each constraint below comes directly from the problem statement or from its security and privacy rules. The agent must build an automated check or a visible UI element for each one.

ID	Constraint	How it is proven
HC-01	Use only the five labels: Low Risk, Suspicious, High Risk, Confirmed Malicious, Inconclusive.	Unit test over all code paths; label is an enumeration, not free text.
HC-02	Never output the words safe, 100% safe, secure, guaranteed or similar as a verdict. Show the standard disclaimer on every report.	Banned-phrase scan on all generated text (AI and templates); disclaimer present in every report.
HC-03	A valid HTTPS certificate never lowers risk. A lookalike domain with a valid certificate is still at least Suspicious.	Test with a lookalike domain that has a valid certificate; scoring has no HTTPS bonus.
HC-04	Absence of a threat-feed match is never treated as proof of safety.	Limitation text added to every report; Low Risk wording says "within the available checks".
HC-05	Confirmed Malicious only via the authoritative-evidence rule in Section 8, never from heuristics or the AI.	Test: strong heuristics alone cannot yield Confirmed Malicious.
HC-06	Urgent or persuasive language alone must not label a message as fraud. Language-only messages are capped (Section 8).	Test with urgent-only messages and legitimate bank alerts.
HC-07	Raw URLs, message text, personal data, emails, passwords and sensitive data stay OFF-CHAIN. The contract stores only hash, timestamp, reporter address, status and revision.	Contract has no string storage; code review; README and UI state this.
HC-08	A blockchain record proves integrity and history of a report, not that the report is true. This is stated in the UI.	Exact copy text present (Appendix F).
HC-09	SSRF protection on every outbound request to a user-influenced host, including every redirect hop, TLS connection and header fetch.	Blocked-address test list in Section 12; visible "Security controls" panel.
HC-10	Passive inspection only: never execute scripts, never download files, cap body size, apply timeouts, rate-limit requests, do not fetch URLs already confirmed malicious.	Code review; tests for size cap and timeouts.
HC-11	AI safety: untrusted input is wrapped in delimiters; the AI cannot set the label; the AI may cite only evidence IDs supplied to it; page content is never sent to the AI.	Prompt-injection test messages in Section 12; validator rejects unknown evidence IDs.
HC-12	eMudhra guardrail: never claim to detect a Certificate Authority's internal key-generation weakness from a URL. Only report a match against a published distrust list using certificate-chain data, citing the source.	Wording template in Phase 3; simulated fixture clearly labelled as simulation.
HC-13	Never ask for or store passwords, OTPs, private keys or seed phrases. The message box warns users. Sensitive-looking tokens are redacted before storage and before any AI call.	Redaction tests; warning text visible.
HC-14	Non-intrusive checks only: no exploitation, no path guessing for secrets, no active vulnerability testing of third-party sites.	Code review: only the final page, robots.txt and security.txt are requested.
HC-15	Secrets (API keys, the wallet private key) live only in backend environment variables; never in the repo, frontend bundle, logs or reports.	Secret scan before each commit; example environment file lists names only.
HC-16	Submitted URLs are never rendered as clickable links in the UI. Display them defanged and escape all untrusted text.	UI inspection test with a script-tag input.

4. Human prerequisites (do these first, in parallel)

These steps need a human with accounts. Start them immediately because several have waiting time. The agent should list which of them it is blocked on, and use the fallbacks noted below if a key is not ready.

#	Task	Why it is needed	Fallback if late
1	Create the public GitHub repository for the team.	Required deliverable and the agent's working base.	None - do first.
2	Create a free abuse.ch account and obtain the Auth-Key used by the URLhaus API.	URLhaus requires this key for API lookups; it is the primary threat feed.	Use the bundled documented dataset of known-bad patterns, and record URLhaus as unavailable.
3	Optional: create a Google Cloud API key with the Safe Browsing API enabled.	Second threat feed for better coverage.	Skip; one feed or documented dataset is enough.
4	Create a Gemini API key in Google AI Studio.	Powers the AI explanation.	Deterministic template explanation is built in as a fallback.
5	Create a brand-new throwaway crypto wallet used only for this hackathon. Never reuse a personal wallet. Put its key only in the backend environment. Fund it with free testnet tokens from a public faucet.	The backend signs the commit transactions with it.	Run the local Hardhat network and document it as the reproducible setup.
6	Get an RPC URL for the chosen testnet (Sepolia preferred; Polygon Amoy alternative) from a free provider.	Lets the backend send and read transactions.	Local Hardhat node.
7	Create a free hosted PostgreSQL database and copy the connection string.	Free backend hosts often lose local files between deploys, so the database must be hosted.	SQLite for local development only.
8	Create Vercel and Render accounts and connect the GitHub repository.	Hosting for frontend and backend.	Run locally and record a demo video.
9	Optional: obtain an explorer API key to verify the contract source publicly.	Nice-to-have for credibility.	Skip.

HUMAN TASK

Testnet faucets are free but can be restrictive: some refuse brand-new empty addresses and all limit how much you can claim per day. Request funds early, try more than one faucet, and keep the local Hardhat setup working as the guaranteed fallback so the demo never depends on a faucet.

NOTE

Free hosting tiers may put the backend to sleep when idle, causing a slow first request. Plan to open the live site a few minutes before any judging session, and make the UI show a friendly "waking up the server" message instead of an error.

5. Fixed technology decisions

These are decided so the agent does not spend time deliberating. Equivalent substitutions are allowed only if a chosen tool proves unusable, and the change must be recorded in the README.

Layer	Decision	Notes
Backend language and framework	Python 3.11 or newer with FastAPI	Async endpoints; background task for scans; typed request/response models.
Frontend	Next.js (App Router) with TypeScript and Tailwind CSS	Single-page dashboard; responsive; no inline HTML injection from untrusted data.
Database	PostgreSQL (hosted) in production, SQLite for local development	Same table design; store the full report as a JSON document column.
AI model	Gemini API, a current fast (Flash-class) model chosen via an environment variable	Structured JSON output; low temperature; short timeout; deterministic fallback.
Threat intelligence	URLhaus (primary), optional Google Safe Browsing, plus a bundled documented dataset	At least one external source or documented dataset is mandatory.
Certificate intelligence	Direct TLS handshake plus crt.sh Certificate Transparency queries	crt.sh can be slow; always time-box it.
Domain intelligence	RDAP for registration age; DNS resolution; public-suffix list for registrable domains	Prefer RDAP over legacy WHOIS.
Smart contract	Solidity 0.8.x with Hardhat for compile, test and deploy; ethers library for the backend	About 30 to 50 lines; no string storage.
Blockchain network	Ethereum Sepolia as primary (Polygon Amoy acceptable); local Hardhat node as guaranteed fallback	Network name and explorer base URL come from configuration.
Hashing	SHA-256 over a canonical serialisation of the report	The 64-hex-character digest is sent to the contract as a 32-byte value.
Hosting	Vercel (frontend), Render (backend), hosted Postgres	Enable a health endpoint and set CORS to the frontend origin only.

Environment variables the system needs (names only; values supplied by the human)

Variable	Used by	Purpose
DATABASE_URL	Backend	Database connection string
GEMINI_API_KEY and GEMINI_MODEL	Backend	AI explanation
URLHAUS_AUTH_KEY	Backend	Threat feed authentication
GOOGLE_SAFE_BROWSING_KEY	Backend (optional)	Second threat feed
CHAIN_RPC_URL, CHAIN_ID, CHAIN_NAME	Backend	Which network to talk to
CONTRACT_ADDRESS	Backend	Deployed registry address
RELAYER_PRIVATE_KEY	Backend only	Signs commit transactions (throwaway wallet)
VALIDATOR_PRIVATE_KEY	Backend only (optional)	Second wallet used by the demo validator to mark Verified or Disputed
EXPLORER_BASE_URL	Backend and frontend	Builds transaction and address links
FRONTEND_ORIGIN	Backend	CORS allow-list
NEXT_PUBLIC_API_BASE_URL	Frontend	Where the API lives (public value, not a secret)
DEMO_MODE	Backend	Enables tamper-simulation routes; off by default
RATE_LIMIT_PER_MINUTE	Backend	Per-IP scan limit

6. Repository layout

Create one repository with three applications and a documentation folder. Folder names are fixed so that the README and test reports can refer to them.

Folder	Contents
/backend/app/api	HTTP routes (Section 10), request validation, error formatting, rate limiting.
/backend/app/core	Configuration loader, logging (without sensitive content), the SSRF guard, the safe HTTP client.
/backend/app/analyzers	URL and domain analyser, TLS analyser, header analyser, threat-intelligence clients, message analyser.
/backend/app/scoring	Signal registry, score calculation, label decision, evidence-coverage calculation.
/backend/app/llm	Prompt construction, redaction, output validation, deterministic fallback explainer.
/backend/app/correlation	Indicator extraction and deterministic campaign matching.
/backend/app/web3	Canonical serialisation, hashing, contract client, commit and verify logic.
/backend/app/db	Tables, access functions, seed loader.
/backend/data	Brand list, keyword lists, suspicious-TLD list, shortener list, message phrase lists, distrusted-CA list, bundled threat dataset (see Appendices).
/backend/tests	Unit and integration tests, including the SSRF blocked-address list and prompt-injection samples.
/contracts	The registry contract, its automated tests, deployment script, and a deployments record holding network, address, deployment transaction and explorer links.
/frontend	Pages, components, API client, display helpers (defanging, escaping), styles.
/fixtures	Cached full reports for the six demo cases and the seeded history used for campaign linking (clearly labelled as demo seed).
/docs	Architecture description and diagram, analysis pipeline, threat-intelligence sources, security methodology, testing report, screenshots.
Repository root	README, example environment file, licence, and a short CONTRIBUTING note for the team.

7. Data contracts

Define these structures once and reuse them everywhere: backend models, frontend types, stored documents, cached fixtures and the hash. Field names below are the exact names to use.

7.1 Scan request

Field	Rules
type	Either url (a link or bare domain) or message (pasted SMS or email text).
value	Required. For url: at most 2,048 characters, printable characters only. For message: at most 5,000 characters; control characters stripped.
use_cached	Optional boolean for demo mode. When true, the matching cached sample is returned and labelled as a cached sample.

7.2 Report object (the full record kept off-chain and hashed)

Field	Type and meaning
report_id	UUID assigned when the scan starts.
schema_version	Integer, starts at 1.
created_at	UTC timestamp in ISO-8601 text, set when analysis completes.
input_type	url or message.
input_display	The submitted value after redaction of sensitive tokens. Stored off-chain only.
normalized_url, registrable_domain, final_url, final_domain	Text values produced by normalisation and redirect following; empty for message-only scans.
resolved_ips	List of public IP addresses the final host resolved to.
risk_label	One of the five labels (an enumeration).
risk_score	Integer 0 to 100 (internal measure; never shown as a percentage of danger).
evidence_coverage	Object with an integer percent, the list of completed check groups, and the list of unavailable groups.
findings	List of Finding objects (7.3), sorted by severity then points.
checks	List of check records: check_id, status (passed, flagged, unavailable, skipped), duration_ms, short note.
redirect_chain	Ordered list of hops: url (defanged for display), status code, registrable domain, whether the hop stayed on the same site.
tls	Certificate report: subject, SAN list, issuer, validity dates, days remaining, public key algorithm and size, signature algorithm, certificate SHA-256, public-key SHA-256, chain list, trust result, CT summary, TLS verdict (see Phase 3).
security_headers	List of header results: header name, present or missing, observed value, severity.
threat_intel	Which feeds were queried, which answered, which were unavailable, and every match with source name, listing status and date.
campaign	linked (boolean), strength (strong or possible), matched_indicators, related_report_count, first_seen, cluster_size.
message_analysis	Present only for message scans: extracted URLs, emails, phone numbers (hashed for storage in indicators), UPI IDs, signals triggered, per-URL labels.
ai_explanation	summary text, evidence_points (each tied to a finding id), recommended_actions, limitations_note, model_name, generated_by (llm or fallback_template).
recommended_actions	Ordered list of plain-language actions.
limitations	List of plain statements, including every unavailable check and the standard feed-limitation sentence.
disclaimer	The exact standard disclaimer from Appendix F.

Field	Type and meaning
analysis_duration_ms	Integer.
is_cached_sample	Boolean; true only for fixtures.
web3	Object added after commit: status (not_committed, pending, committed), report_hash, tx_hash, block_number, network, chain_id, contract_address, committed_at, revision. This object is NOT part of the hash.
report_hash	SHA-256 text value computed at commit time. NOT part of the hash.

7.3 Finding object

Field	Meaning
id	Unique within the report, for example F-03. The AI may cite only these ids.
signal_id	Identifier from the scoring tables in Section 8 (for example IMP-01).
category	IMPERSONATION, STRUCTURE, DOMAIN, REDIRECT, TLS, INTEL, HEADERS or CONTENT.
title	Short plain title.
description	One or two sentences in plain language.
severity	Info, Low, Medium, High or Critical.
points	Integer points contributed to the score before caps.
technical_evidence	Key-value facts that prove the finding (for example: matched brand, distance, certificate dates).
source	Where the fact came from: lexical analysis, RDAP, DNS, TLS handshake, crt.sh, URLhaus, bundled dataset, header inspection, message rules.
potential_impact	What the issue could mean for a user or site owner.
recommendation	What to do.
limitation	What this finding does not prove.

7.4 Check groups and their evidence-coverage weights

Check group	What it covers	Coverage weight
Lexical analysis	Normalisation, brand and homograph checks, URL structure	20
Domain and DNS	RDAP age and DNS resolution	15
Redirect and fetch	Redirect chain and final response headers	15
TLS	Handshake, certificate, chain, key strength, distrust-list match	20
Security headers	Header inspection of the final response	5
Threat intelligence	At least one feed or the bundled dataset answered	20
Certificate Transparency	crt.sh lookup	5

Evidence coverage is the sum of weights for groups whose status is passed or flagged, divided by 100 and shown as a percent. Groups that were unavailable or skipped do not count and are listed under Limitations.

8. Scoring and decision rules

The scoring engine is deterministic. It is the only component that decides the risk label. Each triggered signal becomes a finding with points. Points are summed per category, each category is capped, and the capped totals are added to give the risk score (maximum 100).

8.1 URL signal table

Signal	Category	Condition	Points
IMP-01	IMPERSONATION	Registrable domain contains a known brand keyword but is not an official domain of that brand.	40
IMP-02	IMPERSONATION	Second-level label is within edit distance 1 or 2 of a brand label (distance 1 for brand labels shorter than 5 characters), or matches after confusable-character normalisation (zero for o, one for l or i, rn for m, vv for w).	40
IMP-03	IMPERSONATION	Brand keyword appears only in a subdomain or path of an unrelated registrable domain.	30
IMP-04	IMPERSONATION	Internationalised or punycode domain with characters that visually mimic Latin letters, or mixed scripts within one label.	40
IMP-05	IMPERSONATION	(Messages) the organisation named in the text does not match the domain of the included link.	25
STR-01	STRUCTURE	Phishing keyword (Appendix B) in the hostname together with a brand keyword.	15
STR-02	STRUCTURE	An IP address is used as the host.	20
STR-03	STRUCTURE	A user-info section (the at-sign trick) appears before the real host.	20
STR-04	STRUCTURE	Four or more subdomain levels, or three or more hyphens, or hostname longer than 60 characters.	10
STR-05	STRUCTURE	Suspicious top-level domain (Appendix C).	8
STR-06	STRUCTURE	Known URL-shortener host.	10
STR-07	STRUCTURE	Query parameter carries another URL (embedded redirect).	10
DOM-01	DOMAIN	Domain registered less than 30 days before analysis.	25
DOM-02	DOMAIN	Domain registered 30 to 90 days before analysis.	10
DOM-03	DOMAIN	Domain does not resolve in DNS (also lowers coverage).	5
RED-01	REDIRECT	Redirect ends on an unrelated registrable domain.	20
RED-02	REDIRECT	More than three redirect hops.	5
RED-03	REDIRECT	Redirect downgrades from HTTPS to HTTP.	10
TLS-01	TLS	Certificate expired or not yet valid.	25
TLS-02	TLS	Hostname not covered by the certificate's SAN list.	30
TLS-03	TLS	Chain not trusted: self-signed, unknown issuer or incomplete chain.	25
TLS-04	TLS	Weak public key (RSA below 2048 bits or elliptic curve below 256 bits).	15
TLS-05	TLS	Weak signature algorithm (SHA-1 or MD5).	15
TLS-06	TLS	A certificate in the presented chain matches, by fingerprint, an entry in the distrust list.	45 and floor of High Risk
TLS-06n	TLS	Issuer name only matches a distrusted CA family (no fingerprint match).	15 (advisory)
TLS-07	TLS	Certificate issued less than 7 days ago (weak signal; free certificates are normal).	3
TLS-08	TLS	CT anomaly: for an established domain (over one year old or with more than 5 earlier certificates), a certificate from a never-before-seen issuer was issued in the last 7 days.	10 (advisory)

Signal	Category	Condition	Points
INT-01	INTEL	Exact URL, or host with online status, listed by an authoritative feed.	Forces Confirmed Malicious
INT-02	INTEL	Host listed historically or with offline status in an authoritative feed.	40
INT-03	INTEL	Match in the bundled documented dataset.	40
HDR-xx	HEADERS	Each missing recommended security header (Appendix D).	2 each, category capped at 8; not counted as an independent category

8.2 Category caps and mitigations

- Category caps: IMPERSONATION 50, STRUCTURE 35, DOMAIN 30, REDIRECT 25, TLS 50 (TLS-06 excepted), INTEL 40 (INT-01 excepted), HEADERS 8.
- Mitigation MIT-01: if the registrable domain exactly equals an official domain in the brand list, impersonation signals do not apply; if no other signal fires the result may be Low Risk. A mere subdomain of an unrelated domain never qualifies.
- Mitigation MIT-02: domain older than five years subtracts 10 points. The score never goes below 0, and this never offsets INTEL signals or TLS-06.
- A valid HTTPS certificate gives no mitigation of any kind (HC-03).

8.3 Label decision, in this exact order

1. If INT-01 fired: label is Confirmed Malicious. Stop. (Evidence: the feed name, listing status and date.)
2. Compute the capped score and the number of distinct flagged categories, excluding HEADERS.
3. If TLS-06 fired: label is at least High Risk. It is never Confirmed Malicious on its own, because an untrustworthy certificate chain does not prove the website itself is malicious. The TLS verdict becomes "Potentially compromised trust".
4. If score is 45 or more AND at least two distinct categories flagged: High Risk.
5. Else if score is 15 or more, OR any IMPERSONATION signal fired (this includes score of 45 or more from a single category, the false-positive control): Suspicious.
6. Else (score below 15): if evidence coverage is 60 percent or more and at least one of the TLS group or the redirect and fetch group completed, Low Risk; otherwise Inconclusive.
7. If coverage is below 50 percent and the label would be Low Risk, the label is Inconclusive.
8. If the host cannot be resolved or reached and no impersonation or structure signal fired, the label is Inconclusive.

> **NOTE**
> Design decision: the problem statement asks the system to distinguish ordinary certificate configuration problems from security incidents involving compromised trust. That is why a fingerprint match to the published distrust list produces a High Risk floor and a special TLS verdict, while a plain expired or mismatched certificate only adds ordinary points. Neither is ever described as a hack of the CA.

8.4 Message signal table

Signal	Condition	Points
MSG-01	Urgency or time pressure (for example within hours, immediately, last chance).	10
MSG-02	Threat of account blocking, closure, penalty or legal action.	10
MSG-03	Request for sensitive data (OTP, PIN, password, PAN, Aadhaar, card details, CVV) or a KYC or account-verification update. Negation guard: phrases that warn against sharing (for example do not share, never share) cancel this signal.	25
MSG-04	Request for payment, fee, UPI collect request, crypto transfer or gift cards.	15
MSG-05	Too-good-to-be-true offer: prize, cashback, refund, guaranteed returns, or a job or internship that requires a fee.	15

Signal	Condition	Points
MSG-06	Impersonation of a bank, government body or company where the sender or link does not match (also raises IMP-05).	25
MSG-07	Remote-access or technical-support lure (asks to install remote-control apps).	20
MSG-08	Shortened or deliberately obfuscated link.	10
URL-derived	Each extracted URL is scanned. A link labelled Suspicious adds 15; High Risk adds 35; Confirmed Malicious forces Confirmed Malicious for the message.	15 / 35 / forces

8.5 Message label decision

1. If any extracted URL is Confirmed Malicious: the message is Confirmed Malicious (evidence is the link's feed match).
2. If the text is shorter than 15 characters or contains nothing analysable: Inconclusive.
3. If no signal fired and no extracted link is Suspicious or worse: Low Risk (within the available checks).
4. If any signal fired: at least Suspicious (the brief defines Suspicious as indicators that need further investigation).
5. Technical corroboration means: an extracted link labelled Suspicious or worse, or a verified domain mismatch from MSG-06, or a threat-intelligence match. Without technical corroboration the label is capped at Suspicious, with one exception below.
6. Exception: signals from three or more different MSG types that include MSG-03 or MSG-04 may reach High Risk, and the report must state that the assessment is language-based and is not verified evidence of fraud.
7. With technical corroboration and a score of 45 or more, the label is High Risk.

HARD RULE
HC-06: urgent or persuasive language alone must never be called fraudulent. The report must always separate suspicious language (what the text says) from verified evidence (what technical checks found).

9. Phased build plan

Estimated effort is shown per phase assuming the agent works while the four team members handle accounts, review and demo preparation. Total about 30 hours; Section 13 shows how to schedule and what to cut if behind.

Phase 0 - Foundations (about 1.5 hours)

Goal. A running skeleton of backend, frontend and database that agrees on the data contracts, so every later phase plugs in.

Steps

1. Create the repository layout from Section 6, a README stub, an example environment file listing the variable names from Section 5, and a .gitignore that excludes environment files, build output and local databases.
2. Backend: create the web application with a health endpoint, a configuration loader that reads the environment and fails loudly when a required variable is missing (optional ones may be absent), CORS limited to the frontend origin, and logging that never writes submitted messages, full reports or keys.
3. Define typed models for the scan request, report, finding, check record, campaign, ai_explanation and web3 objects exactly as in Section 7. Create matching frontend types.
4. Database: create three tables. scans holds report_id (primary key), created_at, input_type, risk_label, risk_score, the full report document, the canonical serialisation text (empty until commit), report_hash, and flags for committed and demo-seed. indicators holds an id, report_id, indicator type, indicator value, strength class and created_at, with an index on type plus value. commits holds report_id, tx_hash, block_number, network, chain_id, contract_address, revision, committed_at.
5. Frontend: create the dashboard shell with a header, two tabs (Check a Link, Check a Message), a quick-test panel with six placeholder buttons, and an empty results area. Connect it to the health endpoint and show a friendly waiting message if the backend is slow.
6. Add a small in-process job runner so that starting a scan returns immediately with a report id while analysis continues in the background and progress can be polled.

Acceptance checks (all must pass before moving on)

- The health endpoint answers; the frontend loads and reaches it; tables exist in the database.
- Starting a scan with any input returns a report id and a polling endpoint that reports progress (placeholder analysis is fine).
- No secret appears in the repository (run a secret scan).

Common pitfalls to avoid

- Skipping the shared data contracts and letting frontend and backend drift.
- Using a free-host local file database in production.

Phase 1 - SSRF guard and safe HTTP client (about 1.5 hours)

> **HARD RULE**
> Build this before any code that contacts a user-supplied host. Every later phase must call it. HC-09 and HC-10 live here.

Goal. One reusable gatekeeper and one safe client through which every outbound request to a user-influenced host must pass.

Steps

1. URL validation: accept only http and https schemes. Require a non-empty host. Reject URLs longer than 2,048 characters or containing control characters. Parse the real host using a standards-compliant parser so that tricks such as a trusted name placed before an at-sign are handled correctly; the user-info section is never treated as the host.
2. Port rule: only ports 80, 443, 8080 and 8443 are allowed.
3. Name resolution rule: resolve every address (IPv4 and IPv6) the hostname maps to. Every address must be a public, globally routable address. Reject loopback, private ranges (10.x.x.x, 172.16 to 172.31, 192.168.x.x), link-local (including the cloud metadata address 169.254.169.254), carrier-grade NAT (100.64.0.0/10), multicast,

reserved and unspecified addresses, and the IPv6 equivalents including loopback, unique-local and link-local ranges and IPv4-mapped IPv6 addresses. Reject the host name localhost and similar. Numeric tricks (decimal, octal, hexadecimal or shortened IP forms) must be normalised through the resolver before checking.

4. Anti-rebinding rule: after validating the addresses, connect to the validated address itself rather than resolving again, while still sending the original host name for the Host header and for TLS server-name indication. This prevents a name from changing its answer between check and use.
5. Redirect rule: never let the HTTP library follow redirects automatically. Follow them manually, at most five hops, and run the complete validation above on every hop, including scheme and port. Record each hop (status code, destination) for the report.
6. Request limits: GET and HEAD only; connect timeout about 4 seconds; total per-request timeout about 10 seconds; read at most 200 kilobytes of the body and abort after that; do not store cookies; never execute scripts; if the response is an attachment or an unexpected binary type, abort without reading the body.
7. Scan-level limits: an overall deadline of about 25 seconds per scan, after which unfinished checks are marked unavailable; a per-client rate limit (for example 10 scans per minute per IP) and a limit on simultaneous scans.
8. Identify the scanner honestly in the User-Agent text and never impersonate a browser.
9. Malicious-URL short-circuit: if a threat feed has already confirmed an exact URL as malicious, do not fetch its page at all. Run only name resolution, TLS and lexical checks. (Phase 4 triggers this.)
10. Apply the guard to the TLS handshake and to every other outbound call that targets a user-influenced host. Calls to fixed, trusted services (the threat feeds, RDAP, crt.sh, the AI service, the blockchain RPC) do not need address filtering but must still have timeouts.
11. Expose a description of these controls through a static endpoint so the UI can display a Security controls panel.
12. When validation blocks a request, return a clear error to the user that names the rule that blocked it, without echoing any internal detail.

Acceptance checks (all must pass before moving on)

- All blocked-address test inputs in Section 12 are rejected before any network connection is attempted.
- A redirect that points to a private address is blocked at that hop and the hop is shown in the report or error.
- A very large response is cut off at the size limit; a slow server triggers the timeout.
- The security-controls endpoint returns the list of active protections.

Common pitfalls to avoid

- Validating only the first URL and not each redirect hop.
- Resolving the name a second time after validation (rebinding).
- Forgetting IPv6 and mapped addresses.

Phase 2 - URL and domain analyser (about 3 hours)

Goal. Turn a submitted URL into normalised facts and lexical, domain and redirect findings, each with evidence.

Steps

1. Normalisation: trim whitespace; if no scheme is present assume https; lower-case the host; convert international domains into both their readable and punycode forms; drop the fragment for analysis but keep the original for display. Use a maintained public-suffix list to separate subdomain, registrable domain and suffix. Keep the query parameters as a list.
2. Brand matching: load the brand list (Appendix A). A brand entry has a display name, keywords, one or more official registrable domains and optional aliases. If the submitted registrable domain exactly equals an official domain, mark it as an official match (MIT-01).
3. Impersonation checks (IMP-01 to IMP-04): brand keyword inside an unofficial registrable domain; small edit distance from a brand label; confusable-character normalisation (letters replaced by look-alike digits or letter pairs); brand keyword only in a subdomain or path of an unrelated domain; punycode or mixed-script labels containing characters that mimic Latin letters. Record exactly which brand, which rule and which characters or distance triggered the finding.
4. Structure checks (STR-01 to STR-07): phishing keyword combined with a brand keyword; IP address as host; at-sign trick; deep subdomains, many hyphens or very long host; suspicious top-level domain; URL shortener;

query parameters that contain another URL or common redirect names (url, redirect, next, continue, dest, target).

5. Domain intelligence: query RDAP for the registration date and registrar. Compute age in days. Resolve DNS for the host (addresses, name servers) and store the resolved public addresses as indicators. If RDAP is unavailable, record the check as unavailable rather than guessing. Missing registration data is informational only.
6. Redirect analysis: use the safe client from Phase 1 to follow the redirect chain. Classify each hop as same site, related brand subdomain, or unrelated destination. Raise RED-01 to RED-03 as defined in Section 8. For shortener URLs, resolving the chain is mandatory so the real destination is analysed.
7. False-positive control: the official-domain allow-list, the single-category cap in Section 8, and careful keyword matching (match whole labels or clearly delimited tokens, not arbitrary substrings) must prevent innocent domains from being flagged. Record these decisions in the finding evidence when they apply.
8. Each finding must fill all seven required fields from Section 7.3, using the plain-language text library in Appendix D and the wording rules in Appendix F.

Acceptance checks (all must pass before moving on)

- Inputs in the Section 12 matrix produce the expected signals (for example a bank-keyword-plus-update lookalike fires IMP-01 and STR-01; a punycode brand look-alike fires IMP-04; an official domain fires none).
- A bare domain without a scheme is accepted and normalised.
- A shortened link produces a redirect chain and the final domain is analysed.
- If RDAP and DNS both fail, the checks are marked unavailable and coverage drops accordingly.

Common pitfalls to avoid

- Substring matching that flags harmless domains.
- Treating a young domain as malicious without any other signal (it should only add points).

Phase 3 - TLS certificate and security-header analysis (about 3 hours)

Goal. A certificate security report and a header report that are accurate, evidence-based and honest about their limits.

Steps

1. Handshake: connect through the Phase 1 guard to the host on its HTTPS port. Perform two handshakes. The first uses strict verification against the operating system's trusted roots to learn whether the chain is trusted and, if not, why. The second does not enforce verification, so the certificate and chain as presented can still be captured and analysed even when they are broken.
2. Parse every certificate captured (leaf and intermediates) with a certificate-parsing library, not by hand. Extract subject, SAN list, issuer organisation and name, serial number, start and end dates, days remaining, public-key algorithm and size or curve, signature algorithm, the SHA-256 fingerprint of the whole certificate, the SHA-256 of the public key information, whether it is a CA certificate, whether revocation endpoints are listed, and whether embedded Certificate Transparency timestamps are present.
3. Checks: validity window and soon-to-expire warning (under 14 days); hostname versus SAN including wildcard rules; trust result (trusted, self-signed, unknown issuer, incomplete chain); key strength; signature algorithm; chain configuration (missing intermediate or wrong order); free-certificate-authority use is informational only and never raises risk (HC-03).
4. Revocation: report what revocation information the certificate offers. Only report revoked or not revoked if an actual status check succeeded; otherwise report "revocation status not verified" and add it to Limitations. A live status check is optional and is among the first things to cut.
5. Distrust list check (TLS-06): load the distrusted-CA list (format in Appendix A). Compare every certificate in the presented chain by certificate fingerprint and by public-key fingerprint against list entries. A fingerprint match fires TLS-06. A match only on the issuer name fires TLS-06n and must be described as a possible match needing confirmation.
6. Certificate Transparency: query the crt.sh service for certificates logged for the domain with a time limit of about 8 seconds. Summarise the number of certificates, issuers seen, and certificates issued in the last 30 days, and evaluate TLS-08 only for established domains. If the service is slow or unavailable, mark the CT check unavailable. Never claim a certificate is absent from CT logs unless the lookup succeeded.
7. TLS verdict: assign one of four values and show it in the UI: Valid configuration; Configuration problems; Suspicious; Potentially compromised trust. Map them as follows. Potentially compromised trust only when TLS-06

fired. Suspicious when TLS-02 or TLS-03 fired together with any other signal. Configuration problems when any other TLS signal fired. Otherwise Valid configuration. In every case display: A valid HTTPS certificate only means the connection is encrypted and the certificate matches the domain; it does not mean the website is trustworthy.

8. eMudhra wording rule: when TLS-06 fires, the finding text must say that the presented chain includes a certificate authority certificate that appears on a published distrust list, give the list name, source link and date, state that this means the certificate's trust may be compromised, and state that this tool did not and cannot independently verify any internal key-generation weakness. It must never say the tool detected, hacked or inspected the CA's key generation.
9. Distrust list contents: the human or agent compiles entries from public sources such as the browser root-program incident reports, the certificate database that browsers publish, and published researcher write-ups about the October 2026 eMudhra intermediate-CA incident. Record the source name, link and date for each entry. If fingerprint values cannot be verified from a source, store the entry as unverified; unverified entries may only trigger TLS-06n.
10. Because real sites that chain to those certificate authorities may no longer exist or be reachable, include one clearly labelled SIMULATED FIXTURE that mimics an eMudhra-style distrusted chain so the demo can show the behaviour. The UI must label it as a simulation and never present it as a live finding.
11. Security headers: perform a single request to the final URL through the safe client and inspect response headers. Check presence and quality of Strict-Transport-Security (look at max-age and subdomain coverage), Content-Security-Policy, frame protection (X-Frame-Options or the frame-ancestors directive), X-Content-Type-Options, Referrer-Policy and Permissions-Policy. Also check whether plain HTTP redirects to HTTPS, and note server-version disclosure as informational.
12. Passive endpoint inspection: the only extra paths requested are the robots file and the security-contact file, once each. Do NOT request paths such as environment files, source-control folders or admin panels, and do not guess paths (HC-14).
13. Optional if time remains: a light passive scan of the already-downloaded page text for resource links that start with http while the page is https (mixed content). Version-based vulnerability matching is out of scope unless everything else is finished; if done, use a public vulnerability database, cache results, and label them version-based and unverified.
14. Every header or TLS finding must carry the six items the problem statement requires: the issue, technical evidence, potential impact, severity, recommended remediation and relevant limitation.

Acceptance checks (all must pass before moving on)

- The expired, wrong-host and self-signed test hosts from the public certificate test service (listed in Section 12) each yield the matching TLS signal.
- A well-known healthy site yields Valid configuration, and the standard HTTPS note is displayed.
- A site with missing headers yields header findings that are low in points and never push a clean site above Low Risk.
- The simulated distrusted-chain fixture produces TLS-06 with the exact wording rule and a visible SIMULATION label.
- When crt.sh times out the report shows the CT check as unavailable and the scan still completes.

Common pitfalls to avoid

- Using only the verified handshake, which hides the certificate when verification fails.
- Describing revocation or CT results that were never actually checked.
- Any wording that implies the tool inspected the CA's internals.

Phase 4 - Threat intelligence (about 2 hours)

Goal. At least one working external threat source plus a documented fallback dataset, queried safely and interpreted conservatively.

Steps

1. URLhaus client: query the host and the exact URL using the authentication key sent as a request header. Time-limit each call to about 5 seconds. Parse whether the entry is listed, its status (online or offline), date added, tags and any associated threat label.

2. Optional second feed: Google Safe Browsing lookup for the URL, if a key is available. Treat any unavailable feed as unavailable, never as clean.
3. Query every distinct registrable domain seen in the redirect chain, not only the final one.
4. Interpretation rules: exact URL or an online host listing is INT-01 and forces Confirmed Malicious. A historical or offline host listing is INT-02. A match in the bundled dataset is INT-03. Always show the feed name, listing status and date as the source of the finding.
5. Limitation sentence: URLhaus focuses on malware-distribution URLs and does not cover every phishing site. Add to every report: "Absence of a match in these sources does not prove a site is safe."
6. Bundled documented dataset: create a small curated file of known lookalike and phishing patterns and sample indicators, with a companion document naming each source, its licence and the date it was compiled. This satisfies the requirement for a documented dataset and acts as the fallback when feeds are unreachable.
7. Short-circuit: when INT-01 fires on the exact URL, set a flag so the fetch and header steps are skipped and marked skipped (HC-10).
8. Cache feed answers for a short period to reduce latency and respect rate limits.
9. Testing without touching live malicious sites: unit-test the interpretation logic with recorded sample responses stored in the repository.

Acceptance checks (all must pass before moving on)

- A recorded sample response for an online listing produces INT-01 and a Confirmed Malicious label with the source shown.
- A recorded offline-listing response produces INT-02 only.
- With the feed key removed, the scan completes, the feed is listed as unavailable, and coverage drops.
- No live malicious page is ever fetched during any test.

Common pitfalls to avoid

- Treating a failed lookup as a clean result.
- Fetching a page that a feed has already confirmed as malicious.

Phase 5 - Scoring engine and report assembly (about 2 hours)

Goal. A deterministic component that turns findings into the score, label and coverage, and assembles the complete report.

Steps

1. Create a registry of every signal in Section 8 with its id, category, default severity, points, and text library entry.
2. Implement the cap logic, mitigations, label order (8.3), and the evidence-coverage calculation from Section 7.4.
3. Assemble the report object: sort findings by severity then points; list every check with status and duration; gather recommended actions from findings in priority order; collect limitations from every unavailable or skipped check plus the standard feed-limitation sentence; add the timestamp and the disclaimer.
4. Recommended actions must be concrete, for example: do not open the link or enter any details; contact the organisation using a number from its official website; report the message to the relevant authority or bank; run a security check if details were already entered. The wording must never claim anything is safe.
5. Add a banned-phrase check over all generated text (safe, 100% safe, secure, guaranteed, completely safe, no risk) and fail the build of a report if one appears.
6. Make the engine pure and fully unit-testable from a list of findings, so that Section 12 label tests run without any network.

Acceptance checks (all must pass before moving on)

- Unit tests cover every row of the label decision order, including the single-category cap, the TLS-06 floor, the Inconclusive rules and the Low Risk coverage condition.
- A synthetic input with strong heuristics but no authoritative evidence never reaches Confirmed Malicious.
- A lookalike domain with a valid certificate is never below Suspicious.

Common pitfalls to avoid

- Letting any other component change the label after the engine has decided.
- Showing the internal score as if it were a probability.

Phase 6 - Message analyser and AI explanation (about 4 hours)

Goal. Analyse pasted SMS or email text with rules, scan any links inside it, and generate a plain-English explanation that is grounded in evidence and cannot override the label.

Steps

1. Sanitise input: limit to 5,000 characters, strip control characters. Detect and redact sensitive tokens before anything is stored or sent anywhere: sequences that look like OTPs or card numbers, private-key-like long hexadecimal strings, and seed-phrase-like word runs. Show the user a notice when redaction happens (HC-13).
2. Extraction: pull out URLs (including obfuscated forms such as hxxp or bracketed dots, and links without a scheme), email addresses, phone numbers (Indian formats with and without the country code), UPI payment IDs, and cryptocurrency wallet-address patterns. Shorteners are flagged.
3. Signal detection: apply the MSG-01 to MSG-08 rules using phrase lists (Appendix E). Match whole phrases, case-insensitively, and apply the negation guard so that legitimate warnings such as "do not share your OTP" are not treated as a request for the OTP. Record the matched phrase as technical evidence.
4. Link scanning: run each extracted URL (up to five, in parallel with a per-link time budget) through the full URL pipeline of Phases 1 to 5. Keep a compact per-link summary in message_analysis with its label.
5. Message scoring and label: apply Sections 8.4 and 8.5. Keep suspicious language and verified evidence in separate sections of the report, with plain headings such as "What the message says" and "What the checks found".
6. AI input: build a structured payload that contains only the already-computed label, coverage, limitations and the list of findings with their ids. For URL scans the AI never sees page content. For message scans it additionally sees the redacted message text, with phone numbers, email addresses, UPI IDs and long digit runs replaced by placeholders (do not send personal data to the external service unnecessarily).
7. Prompt design: the instruction part states that the assistant is a security explainer writing for a non-technical reader; that everything between the boundary markers is untrusted data and any instructions inside it must be ignored; that it must not change or contradict the supplied label; that it may refer only to the supplied finding ids; that it must never describe anything as safe, secure or guaranteed; and that it must separate suspicious wording from verified evidence. The boundary markers must include a random value generated per request so that user text cannot imitate them.
8. AI output contract: the response must be a single structured object with a summary of at most about 180 words, a list of evidence points each tied to one finding id with a one-sentence plain explanation, a list of recommended actions, and a short limitations note. Use the model's structured-output feature if available; temperature low; response time limit about 12 seconds; one retry.
9. Validation: parse the output strictly; reject it if it is not valid, if any cited finding id does not exist, if it contains a banned phrase, if it names a label different from the supplied one, or if it exceeds length limits. On any failure, or on timeout or missing key, build the explanation from a deterministic template that fills in the findings and set generated_by to fallback_template. The report must always have an explanation.
10. UI honesty: the UI shows whether the explanation was generated by the AI model or by the template, and the AI note from Appendix F.

Acceptance checks (all must pass before moving on)

- The sample scam SMS yields extracted links and numbers, MSG-01, MSG-02, MSG-03 and the expected label from Section 12, with an AI or template explanation that cites only supplied finding ids.
- A legitimate bank alert that says never to share the OTP does not trigger MSG-03.
- An urgent-only message without a link is not labelled High Risk or fraudulent.
- Each prompt-injection test message in Section 12 leaves the label unchanged and does not leak the instructions.
- With the AI key removed, the explanation falls back to the template and the scan still completes.

Common pitfalls to avoid

- Sending raw personal data to the AI service.

- Letting the AI return its own label that the UI then displays.
- Trusting the AI output without validating ids.

Phase 7 - Persistence and campaign correlation (about 2.5 hours)

Goal. Save every scan and deterministically connect related scans. AI may summarise a connection but never creates one.

Steps

1. On scan completion save the full report in the scans table and write its indicators. Indicator types: final_domain, registrable_domain, resolved_ip, redirect_hop_domain, cert_sha256, cert_spki_sha256, message_template_hash, message_link_domain, impersonated_brand (a pattern tag), and hashed values for phone numbers, UPI IDs and email addresses (store a salted SHA-256 so records can be matched without keeping the raw personal values).
2. Message template hash: lower-case the text, replace every URL, number, email address and phone number with placeholders, collapse whitespace, then hash the result. Two messages that differ only in the link or amount then match.
3. Matching, before the scan finishes: search earlier reports whose label was Suspicious, High Risk or Confirmed Malicious (never Low Risk) for any matching indicator.
4. Strength rules: strong keys are cert_spki_sha256, message_template_hash, hashed payment or phone indicators, and final_domain. A match on any strong key, or on two or more different indicator types, is a strong link and shows the badge "Linked to Existing Phishing Campaign". A single resolved_ip match alone (shared hosting is common) is only a possible link and shows a softer badge "Possible infrastructure overlap".
5. Output: fill the campaign block with linked, strength, matched indicators (domains and addresses shown in defanged form; hashed types shown as descriptions such as "matching message template"), related report count, first seen date and cluster size.
6. AI summary of a link (optional): after the deterministic match, the AI may write one sentence describing the relationship using only the matched indicator list. The UI labels it "AI-summarised; link established by deterministic matching".
7. Older reports are never changed when a newer report links to them, because that would break their hashes. A separate endpoint computes related reports on demand.
8. Seed data: create six to ten synthetic historical reports labelled as demo seed so the badge can be demonstrated, for example two look-alike bank domains that share a hosting address or certificate fingerprint or the same message template. Seed rows carry a visible demo-data marker and are described as such in the README.

Acceptance checks (all must pass before moving on)

- Scanning the second seeded look-alike shows the strong campaign badge with the matching indicator named.
- Two unrelated sites that happen to share a common hosting address show at most the soft badge.
- Re-verifying an older report after a newer scan still returns MATCH (older reports were not modified).

Common pitfalls to avoid

- Linking on weak keys only and overstating a campaign.
- Writing matches back into earlier reports.

Phase 8 - Web3 proof registry (about 4 hours)

8.1 The smart contract (described in words)

Create a contract named **ThreatRegistry** in Solidity 0.8.x, roughly 30 to 50 lines. It must contain no text storage of any kind.

Element	Specification
Record per report	Keyed by a 32-byte report identifier. Holds: the 32-byte report hash, the timestamp of the latest commit, the reporter address (the first committer), a status of Reported, Verified or Disputed, and a revision number that starts at 1.

Element	Specification
Commit function	Takes the identifier and the hash. Rejects a zero hash. If the identifier is new, creates the record with status Reported and revision 1. If it exists, only the original reporter may call it; it replaces the hash and timestamp, increases the revision by one and resets the status to Reported.
Status function	Takes the identifier and a new status. Rejects unknown identifiers. Rejects the original reporter, so nobody can validate their own report. Any other address may mark Verified or Disputed.
Read functions	A function that returns the record for an identifier, and a function that tells whether an identifier exists.
Events	One event on every commit carrying identifier, hash, reporter, revision and timestamp. One event on every status change carrying identifier, new status and who changed it. Events preserve the full revision and dispute history.
Privacy	Nothing but hashes, numbers, addresses and a status enumeration is stored or emitted.

8.2 Contract tests, deployment and records

- Automated tests: creating a record; a second address cannot revise it; the reporter can revise and the revision increases; zero hash rejected; reporter cannot set status; another address can set Verified and Disputed; unknown identifier rejected; events emitted with the right values.
- Deploy first to the local Hardhat network and confirm the whole flow works. Then deploy to Sepolia (or Amoy) using the funded throwaway wallet.
- Write the network name, chain id, contract address, deployment transaction hash and explorer links into a deployments record file in the repository and into the README. Optionally verify the source on the explorer.

8.3 Report hashing procedure (must be implemented exactly)

1. Finish the report: all analysis done, report identifier assigned and creation time set.
2. Remove two fields from a working copy: the web3 object and the report_hash field.
3. Serialise the working copy to text deterministically: UTF-8; object keys sorted alphabetically at every level of nesting; no spaces or line breaks between elements; arrays keep their order; timestamps as ISO-8601 UTC text; no floating-point numbers anywhere (scores, percentages and durations are whole numbers; anything fractional becomes text).
4. Compute SHA-256 over those bytes. The 64-character hexadecimal result is the report hash. It is sent to the contract as a 32-byte value.
5. On-chain report identifier: compute SHA-256 over the report's UUID text and use that as the 32-byte identifier, so no readable identifier is ever published.
6. At commit time, store the exact canonical text in the database next to the report. Verification recomputes from the stored report, compares to the stored canonical text, and compares to the on-chain hash, so library or schema changes later cannot silently alter the result.

8.4 Backend commit and verify behaviour

Operation	Behaviour
Commit (Save and Verify Report)	Idempotent: if the report is already committed, return the existing record and do not send a second transaction. Otherwise compute the hash, send the commit transaction with the backend wallet, wait for one confirmation up to about 60 seconds (if still pending, return the pending state with the transaction hash and let the UI poll), store transaction hash, block number, network, chain id, contract address and revision, update the web3 object, and return the explorer link. Rate-limit commits.
Verify (Verify Integrity)	Reload the report from the database, recompute the hash, read the on-chain record, compare. Return exactly one of: MATCH; MISMATCH (the stored report differs from what was committed); NOT_COMMITTED; NOT_FOUND_ON_CHAIN; CHAIN_UNAVAILABLE. Include recomputed hash, on-chain hash, on-chain timestamp, reporter, status, revision and explorer links.
Status update (optional)	A demo-validator button using a second backend wallet to mark Verified or Disputed, clearly labelled as demo validator, to show the status lifecycle from the problem statement.
Tamper demo (DEMO_MODE only)	Two routes that modify a stored report in a harmless way and restore it from a backup copy kept in a separate column. Disabled unless DEMO_MODE is on. After tampering, Verify must return MISMATCH; after restore, MATCH.

Operation	Behaviour
History	A list endpoint of saved reports showing label, domain (defanged), time, and on-chain status, supporting the problem statement's verifiable reporting history.

HARD RULE
Only the hash, timestamp, status, reporter address and revision go on-chain. The UI and README must state: "Only a cryptographic hash, timestamp, status and reporter address are stored on-chain. Raw URLs, messages and personal data stay in our private database." and "A blockchain record proves a report has not changed since it was recorded. It does not prove the report is true."

Goal. A working on-chain commit and independent verification loop, demonstrated including a failed verification after tampering.

Steps

1. Write and test the contract (8.1 and 8.2).
2. Implement canonical serialisation and hashing (8.3) with unit tests using fixed sample reports whose expected hashes are recorded.
3. Implement the commit and verify operations and the contract client in the backend (8.4).
4. Deploy to local Hardhat, then to the testnet, and record the details.
5. Prepare one fully saved example report whose transaction link is placed in the README.

Acceptance checks (all must pass before moving on)

- Hashing the same report twice, and hashing it after key reordering, gives the same value; changing one character changes the value.
- Commit then Verify returns MATCH. Tamper then Verify returns MISMATCH. Restore then Verify returns MATCH.
- Clicking commit twice does not create two transactions.
- With the RPC unreachable, Verify returns CHAIN_UNAVAILABLE and the UI explains it.
- A transaction link opens the correct transaction on the explorer.

Common pitfalls to avoid

- Letting a floating-point number or key order change the hash.
- Putting the URL or any readable text into any contract call.
- Keeping the wallet key anywhere but the backend environment.

Phase 9 - Frontend dashboard (about 5 hours, can overlap Phases 6 to 8)

9.1 Layout and flow

- Header: product name, a "Security controls active" badge that opens a panel listing the protections (SSRF guard, passive checks only, timeouts and size limits, AI injection guard, off-chain privacy), and a small link to the threat-reports history.
- Two tabs: Check a Link (single input and a Scan button) and Check a Message (large text area with a character counter and the warning "Never paste passwords, OTPs, private keys or seed phrases.").
- Quick-test panel with exactly six one-click buttons in two groups. Group Safe-looking samples: sbi.co.in, github.com, wikipedia.org. Group Suspicious samples: a lookalike bank KYC link, a punycode look-alike of a known brand, and the scam SMS. A small "More tests" menu may hold the certificate test hosts, the simulated distrusted-chain fixture, an SSRF example and a prompt-injection example.
- A toggle "Use cached sample" for demo reliability. When used, every result carries a visible badge "Cached sample result".
- Progress stepper while scanning: Validating link safety, Analysing domain, Following redirects, Inspecting certificate, Checking threat intelligence, Correlating with earlier reports, Writing explanation. Each step updates from the polling endpoint.

9.2 Report view, top to bottom

1. Risk banner: label with colour, icon and text together (never colour alone), a one-line meaning from the label table, analysis timestamp (UTC and local), and an evidence-coverage meter with the list of completed and unavailable

groups.

2. AI explanation card titled "Explanation" with the AI note, the generated-by badge, the summary and the evidence points.
3. Campaign badge when linked, with the matched indicators and cluster size.
4. Key findings table sorted by severity, each row expandable to show technical evidence, source, potential impact, recommendation and limitation.
5. Certificate card: issuer, validity dates and days left, SAN list, key and signature algorithm, chain, trust result, CT summary, TLS verdict, and the standard HTTPS note in a visible banner.
6. Redirect chain as a simple vertical list with domain changes highlighted; all URLs defanged.
7. Threat intelligence panel: each feed queried, its answer, and which were unavailable.
8. Website security findings: header results with severity, impact and remediation.
9. Recommended actions, then Limitations and unavailable checks, then the standard disclaimer.
10. Web3 panel: the two privacy statements, a button Save and Verify Report, then the report hash, transaction hash, network and explorer link; a button Verify Integrity that shows a badge for MATCH, MISMATCH or the other result codes, with recomputed and on-chain hashes side by side; a button to download the report as JSON.
11. In demo mode only: a collapsed panel with Simulate tampering and Restore.

9.3 Quality requirements

- Never render a submitted URL as a clickable link; show it defanged (for example hxxp and bracketed dots) with a copy button. Escape all untrusted text; never insert raw HTML (HC-16).
- Colour palette for labels: Low Risk muted green, Suspicious amber, High Risk orange-red, Confirmed Malicious dark red, Inconclusive grey. The Low Risk badge text must still read Low Risk, never a word like safe.
- Responsive layout that works on a phone-width screen; loading skeletons; clear error messages (blocked by the SSRF guard, server waking up, rate limit reached, AI fallback in use).
- Accessible: keyboard operable, sufficient contrast, labelled inputs, visible focus.
- Optional history page listing saved reports with label, defanged domain, time and on-chain status.

Goal. A clean, reliable interface a judge can use in five seconds.

Steps

1. Build the shell, tabs, quick-test panel and stepper first, wired to the real endpoints.
2. Build the report view in the order of 9.2 and the Web3 panel.
3. Add the security-controls panel, history page (optional), demo-mode panel and the cached-sample toggle.
4. Polish states, copy text (Appendix F) and responsiveness.

Acceptance checks (all must pass before moving on)

- All six demo buttons run end to end and render the full report.
- A script tag typed into the URL box is displayed as inert text.
- A localhost address typed in shows the blocked card with the rule name.
- The label badges and wording never contain the word safe as a verdict.

Phase 10 - Fixtures, tests, documentation and deployment (about 5 hours)

Goal. Make the demo bulletproof, document it, deploy it and submit early.

Steps

1. Run the test matrix in Section 12 against the live pipeline and record observed labels, key evidence and a screenshot for each case.
2. Store a cached full report for each of the six demo cases (and the simulated distrusted-chain case) in the fixtures folder, marked is_cached_sample, used only when the toggle is on or a live dependency fails during judging.
3. Write the documentation set listed in Section 14.
4. Deploy: hosted database first; backend on Render with all environment variables and the health check; frontend on Vercel with the API base URL; set CORS to the frontend origin; run the seed loader once.
5. Run the whole demo script from Section 2 on the production URLs. Fix anything that fails.

6. Record a two to three minute backup video of the full demo in case of network trouble during judging.
7. Freeze features at least six hours before the deadline. Submit at least one hour before 8:00 PM IST on 10 November 2026.

Acceptance checks (all must pass before moving on)

- All items on the submission checklist in Section 14 are complete.
- The production site passes the demo script end to end.
- The README contains the architecture description, contract address, a transaction link and the 3 + 3 test results.

10. API specification

All routes live under /api and exchange JSON. Errors use one consistent shape: a machine code, a plain-language message safe to show users, the name of the rule that blocked the request when applicable, and a request id.

Method and path	Purpose	Notes and errors
GET /api/health	Liveness check for the host.	Always cheap; used by the hosting platform.
GET /api/security-controls	Static description of active protections for the UI panel.	No secrets, no configuration values.
GET /api/samples	The six demo inputs and the optional extra tests.	Each has id, type, value, group (safe-looking or suspicious).
POST /api/scans	Start a scan. Body is the scan request from 7.1.	Returns 202 with report_id. 400 when invalid or blocked by the SSRF guard (names the rule). 429 when rate-limited.
GET /api/scans/{report_id}	Poll progress.	Returns status (queued, running, complete, failed), per-check statuses so far, and the full report once complete. 404 if unknown.
GET /api/reports/{report_id}	Fetch a stored report including its web3 object.	404 if unknown.
GET /api/reports	List saved reports as summaries.	Filters for label and committed status; pagination; never returns raw message text.
GET /api/reports/{report_id}/related	Related reports computed on demand from indicators.	Summaries only; see Phase 7.
POST /api/reports/{report_id}/commit	Hash the report and commit it on-chain (Save and Verify Report).	Idempotent. Returns committed or pending with transaction hash and explorer link. 503 if the chain is unreachable or the wallet has no test funds, with a clear message.
GET /api/reports/{report_id}/verify	Verify Integrity.	Returns one result code: MATCH, MISMATCH, NOT_COMMITTED, NOT_FOUND_ON_CHAIN, CHAIN_UNAVAILABLE, plus both hashes and chain details.
POST /api/reports/{report_id}/status	Optional demo validator marks Verified or Disputed.	Uses the validator wallet; clearly labelled demo.
GET /api/reports/{report_id}/download	Download the stored report as a JSON file.	Includes the web3 object when present.
POST /api/demo/tamper/{report_id} and POST /api/demo/restore/{report_id}	Demo-mode tamper simulation and restore.	Return 404 unless DEMO_MODE is on.

11. Failure handling and graceful degradation

Reliability is graded. Every dependency can fail during judging, so the product must degrade visibly and honestly. The rule is the same every time: record the check as unavailable, list it under Limitations, lower evidence coverage, and continue.

Dependency or condition	Behaviour when it fails
DNS resolution fails	Fire DOM-03, skip fetch and TLS (mark skipped), keep lexical findings, apply the unreachable-host rule in Section 8.3.
RDAP unavailable	Domain-age check marked unavailable; no age signals.
Target slow or refusing connections	TLS, redirect and header groups marked unavailable after their timeouts; scan continues.
URLhaus key missing or service down	Feed listed as unavailable; the bundled dataset still runs; coverage drops.
crt.sh slow or down	CT check marked unavailable after about 8 seconds.

Dependency or condition	Behaviour when it fails
AI service down, key missing, or output invalid	Deterministic template explanation is used and labelled as such.
Overall scan deadline reached	Unfinished checks marked unavailable; a partial report is returned; Inconclusive may result.
Database unreachable	Return a clear service-unavailable message; never fake a saved report.
Blockchain RPC unreachable	Commit returns a clear 503 message; Verify returns CHAIN_UNAVAILABLE.
Relayer wallet out of test funds	Commit returns a clear message asking the team to refill from a faucet; UI explains it.
Backend asleep on free hosting	Frontend shows a waking-up message and retries health checks before showing an error.

12. Test plan

12.1 The mandatory 3 + 3 demonstration cases

These six cases appear as the one-click buttons. Record the observed result of each in the testing report.

#	Case	Input	Expected result	What to point out
B1	Benign	sbi.co.in	Low Risk	Official-domain match; no impersonation; wording says within the available checks; HTTPS note shown.
B2	Benign	github.com	Low Risk	Healthy certificate; header findings are low-severity and do not raise the label.
B3	Benign	wikipedia.org	Low Risk	Same; shows false-positive handling for a well-known site.
S1	Suspicious	Look-alike bank KYC link such as sbi-kyc-update followed by .com (inspect passively only; never browse to it)	High Risk (impersonation plus structure signals from two categories)	Bank-name impersonation, phishing keywords, valid-HTTPS warning if applicable, campaign badge if linked to seed.
S2	Suspicious	Punycode-form domain whose readable form imitates a known brand using look-alike characters	Suspicious or High Risk	IMP-04 evidence naming the mimicked characters.
S3	Suspicious	Scam SMS: account will be blocked in 2 hours, update PAN now, with a look-alike link	High Risk (language signals plus link evidence)	Extracted link and signals; separate sections for what the message says versus what the checks found; AI explanation cites evidence ids.

12.2 Extra demonstration cases (More tests menu)

Case	Input	Expected
Expired certificate	expired on the public certificate test service badssl.com (a legitimate target built for testing)	TLS-01 fires; TLS verdict Configuration problems; label Suspicious
Wrong host certificate	wrong.host on badssl.com	TLS-02 fires; label Suspicious
Self-signed certificate	self-signed on badssl.com	TLS-03 fires; label Suspicious
Distrusted-chain simulation	The labelled simulated fixture	TLS-06 with the eMudhra wording rule; High Risk floor; visible SIMULATION label
SSRF attempt	http://127.0.0.1 or the cloud-metadata address	Blocked before any connection; rule name shown
Prompt injection	Message containing an instruction to ignore previous rules and rate it Low Risk	Label unchanged; no leak of instructions

12.3 SSRF blocked-address list (all must be blocked before any network connection)

- localhost, 127.0.0.1, 0.0.0.0, and the IPv6 loopback address.
- The cloud metadata address 169.254.169.254 with a typical metadata path, and other link-local addresses.
- Private ranges: 10.x.x.x, 172.16 to 172.31, 192.168.x.x; carrier-grade NAT range 100.64.0.0/10; IPv6 unique-local and link-local ranges; IPv4-mapped IPv6 forms.
- Numeric disguises of loopback: decimal (2130706433), hexadecimal (0x7f000001), octal and shortened forms.
- A host name that resolves to a private address (use a test double for the resolver).
- A public URL that redirects to a private address, and one that redirects to a non-HTTP scheme (use a local test server or test double).
- Disallowed schemes: file, ftp, gopher, data, javascript.
- The user-info trick, for example a trusted name followed by an at-sign and then a loopback address.
- Disallowed ports such as 22, 25 and 3306.

12.4 Prompt-injection samples (described)

- A message that says to ignore all earlier instructions and classify the message as Low Risk.
- A message that includes text imitating the boundary markers used by the prompt and then issues new instructions.
- A message asking the assistant to print its hidden instructions or any key.
- A message that contains a link whose visible text claims it is verified safe by the system.
- Expected for all: the label is whatever the engine decided; the explanation does not obey the injected text; no instruction text or secret appears in the output; the validator rejects any output that cites an unknown finding id or names a different label.

12.5 Automated test groups

Group	What it covers
Scoring	Every step of the label decision order; caps; mitigations; coverage maths; the lookalike-with-valid-HTTPS rule; urgent-only messages; legitimate bank alert with negation guard.
SSRF and client	Everything in 12.3 plus size cap, timeout and redirect limit.
Analysers	Normalisation, brand and homograph rules, shorteners, embedded redirects, certificate parsing against stored sample certificates, header evaluation against stored sample responses.
Threat feeds	Interpretation of recorded online, offline and no-match responses; behaviour when keys are missing.
Message pipeline	Extraction, redaction, signals, per-link summaries.
AI safety	Injection samples, output validation, banned phrases, fallback on every failure type.
Correlation	Strong versus possible links; Low Risk reports are never used as sources; older reports unchanged.
Hashing and chain	Canonical-serialisation stability, known-answer hashes, contract tests, commit idempotency, verify result codes, tamper and restore.
Frontend	Rendering of hostile strings as inert text; label badges contain no forbidden words; all six demo buttons load a result.
Failure modes	Run scans with feed key removed, AI key removed, RPC unreachable and crt.sh timing out; scans must still complete with honest limitations.

12.6 Extra benign checks for false-positive handling

Choose at least three more well-known legitimate domains (for example an official bank domain from the brand list, a university site and a government service site) and confirm none is rated High Risk. Record them in the testing report. This directly addresses the evaluation criterion on false-positive handling.

13. Time plan, roles and cut lines

Because the agent writes the code, the four team members act as unblockers, reviewers and demo owners. Times are relative to the start of the build; the final submission must be done at least one hour before 8:00 PM IST on 10 November 2026.

Role	Responsibility
Team lead	Owns Section 4 prerequisites, the wallet and faucet funds, deployment accounts, the final submission and timekeeping.
Security reviewer	Reviews Phases 1 to 4 against the Hard Constraints, runs the SSRF list, checks wording about certificates and the eMudhra rule.
Product and UI reviewer	Reviews Phase 9, copy text and demo flow; confirms no forbidden wording; tests on a phone.
Docs and evidence owner	Collects screenshots, fills the testing report, writes README sections, records the backup demo video.

Time block (hours from start)	Agent work	Human work
0 to 3	Phase 0 and Phase 1	Create repo; request all keys and testnet funds; hosted database
3 to 9	Phases 2, 3 and 4	Compile the distrusted-CA list sources and brand list review; test keys as they arrive
9 to 14	Phases 5 and 6	Review scoring decisions; prepare the scam SMS and test inputs
14 to 19	Phases 7 and 8; Phase 9 in parallel if the tool supports several agents	Check wallet funds; deploy contract to testnet; record details
19 to 24	Finish Phase 9; Phase 10 fixtures and test runs	Run the test matrix; capture screenshots
24 to 28	Deployment, production end-to-end run, fixes	Production demo run; write README and testing report
28 to 30	Buffer, final fixes only	Backup video; submission at least one hour before the deadline

If fewer than 30 hours remain, run independent phases in parallel with several agent sessions: for example security and analysers (Phases 1 to 4), contract and Web3 (Phase 8), and the frontend shell (Phase 9) after Phase 0 has fixed the shared data contracts.

What to cut, in this order, if behind schedule

1. Version-based vulnerability matching and mixed-content scanning.
2. Live revocation status checking (keep the honest 'not verified' wording).
3. The history page and the optional status-validation button.
4. The second threat feed (keep one feed plus the bundled dataset).
5. The AI one-sentence campaign summary.
6. Certificate Transparency anomaly scoring (keep the crt.sh summary or mark it unavailable).

> **HARD RULE**
> Never cut: the SSRF guard, TLS inspection, one working threat source or documented dataset, the AI explanation with fallback, the contract with commit and verify, the six demo cases, the test report and the README. These are the minimum requirements in the problem statement.

14. Documentation and submission

14.1 README outline

1. What the product does, in three sentences, and a screenshot.
2. Architecture overview with a diagram (an image or a text-based diagram block) showing frontend, backend modules, database, threat feeds, AI service, contract and wallet.

3. Analysis pipeline: the stages from Section 9 in order, and which component decides the label.
4. Threat-intelligence sources: names, what each covers and does not cover, and the bundled dataset's origin and licence.
5. Security methodology: SSRF guard, passive-only inspection, limits, AI injection guard, redaction, off-chain privacy rule, and the eMudhra guardrail wording.
6. Web3 section: contract description, contract address, network, deployment transaction link, an example commit transaction link, the hashing procedure in plain words, and the two privacy statements.
7. How to run locally: prerequisites, environment variable names, and the order to start the database, backend, contract node and frontend; how to run the tests.
8. Testing report summary and a link to the detailed document.
9. Known limitations and honest scope boundaries (the system does not guarantee any site is safe or malicious; it is not an antivirus; it does not exploit vulnerabilities).

14.2 Testing report format

Column	Content
Case id and name	B1 to B3, S1 to S3, plus extras.
Input type and input	As submitted (defang URLs).
Expected label	From Section 12.
Observed label and coverage	What the live system returned.
Key evidence	The top findings and their sources.
AI explanation excerpt	A short quote showing it cites evidence.
Screenshot	Of the report view.
Notes	Any deviation and why.

14.3 Submission checklist mapped to the nine deliverables

Deliverable	Evidence to have ready
Working prototype	Live frontend URL and live backend URL; the demo script passes.
Public GitHub repository	Complete source, README, setup instructions, no secrets.
AI demonstration	The scam SMS report with its evidence-grounded explanation.
Cybersecurity demonstration	The look-alike URL report with TLS analysis and the HTTPS warning.
Web3 demonstration	Deployed contract on a testnet, or the reproducible local Hardhat setup.
Verification evidence	Contract address and a transaction link; a MATCH result and a MISMATCH result after tamper demo.
Technical documentation	Architecture, pipeline, sources, security methodology in /docs and the README.
Testing report	3 benign + 3 suspicious cases with observed results.
Live demonstration	Rehearsed run of the Section 2 demo script, plus the recorded backup video.

14.4 Scoring-aware demo talking points

- Say early that the rules decide the label and the AI only explains; this answers the AI-security and prompt-injection criteria.
- Show a lookalike site with a valid certificate still rated at least Suspicious: valid HTTPS is not trust.
- Show the eMudhra-style simulated chain and read the careful wording: the tool matches a published distrust list; it does not inspect CA internals.
- Show the tamper demo: MATCH, then MISMATCH, then MATCH. This is the strongest proof of the Web3 integrity claim.
- Say the sentence: only the hash goes on-chain; raw URLs and messages never leave the database; the chain proves integrity, not truth.

Appendix A - Data files the app needs

A.1 Starter brand list (/backend/data)

Starter set only. A human must review the official domains before the demo and extend the list. Indian banks have been moving to a dedicated bank-domain family, so check each bank's current official domains and add them. An official match requires the exact registrable domain, never a look-alike or an unrelated parent.

Brand	Keywords	Official registrable domains (starter)
State Bank of India	sbi, onlinesbi, statebank	sbi.co.in, onlinesbi.sbi
HDFC Bank	hdfc, hdfcbank	hdfcbank.com
ICICI Bank	icici, icicibank	icicibank.com
Axis Bank	axisbank	axisbank.com
Punjab National Bank	pnb	pnbindia.in
Paytm	paytm	paytm.com
PhonePe	phonepe	phonepe.com
Google	google, gmail	google.com
Microsoft	microsoft, office365, outlook	microsoft.com, office.com, outlook.com, live.com
Amazon	amazon	amazon.com, amazon.in
Flipkart	flipkart	flipkart.com
PayPal	paypal	paypal.com
Apple	apple, icloud	apple.com, icloud.com
Meta platforms	facebook, instagram, whatsapp	facebook.com, instagram.com, whatsapp.com
LinkedIn	linkedin	linkedin.com
GitHub	github	github.com
Income Tax Department (India)	incometax	incometax.gov.in
UIDAI (Aadhaar)	uidai, aadhaar	uidai.gov.in
India Post	indiapost	indiapost.gov.in
IRCTC	irctc	irctc.co.in
NPCI / UPI	npci, bhim	npci.org.in
Netflix	netflix	netflix.com

A.2 Distrusted-CA list format

Field	Meaning
ca_name	Name of the certificate authority or intermediate.
match_type	certificate_sha256, public_key_sha256 or subject_name.
value	The fingerprint or name to compare against the presented chain.
reason	Short plain statement of why it is listed (for example: distrusted by browser root programs after an incident).
source_name and source_url	Where the entry came from. Mandatory.
date_added	When the entry was added.
status	verified (fingerprint confirmed from a source) or unverified (name-level only).

Public sources to compile from: browser root-program incident reports, the certificate database browsers publish, and researcher write-ups about the October 2026 eMudhra intermediate-CA incident. Only verified fingerprint entries may trigger TLS-06; unverified entries may only trigger TLS-06n.

A.3 Bundled threat dataset format

Field	Meaning
indicator_type	domain, url_pattern or label pattern.
value	The indicator.
category	For example banking phishing, delivery scam, job scam.
source and licence	Mandatory; also described in a companion document with the compile date.

Appendix B - Keyword lists

B.1 Phishing keywords (used by STR-01 together with a brand keyword)

verify, verification, update, kyc, secure, security, login, signin, account, banking, refund, reward, prize, claim, wallet, support, helpdesk, otp, unlock, suspend, confirm, billing, payment, invoice, pan, aadhaar.

B.2 Redirect parameter names (used by STR-07)

url, redirect, redirect_uri, next, continue, dest, destination, target, return, returnurl, goto, link.

Match whole tokens delimited by punctuation or label boundaries. Do not match arbitrary substrings.

Appendix C - Suspicious top-level domains and URL shorteners

Treat these as weak signals only. Many legitimate sites use some of them.

Suspicious top-level domains (starter): zip, mov, top, xyz, click, work, support, live, rest, icu, cfd, sbs, buzz, gq, tk, ml, cf.

URL shorteners (starter): bit.ly, tinyurl.com, t.co, is.gd, cutt.ly, rb.gy, shorturl.at, ow.ly, buff.ly, bit.do, tiny.cc, rebrand.ly, t.ly.

Appendix D - Plain-language text library for findings

Every finding needs the issue, evidence, potential impact, severity, remediation and limitation. Use these as the base wording; add the actual evidence values from the scan.

Finding	Severity	Potential impact	Recommended remediation	Limitation
Missing Strict-Transport-Security	Medium	Browsers may connect over unencrypted HTTP first, exposing users to interception on hostile networks.	Send the header with a long max-age (at least six months) over HTTPS and consider covering subdomains.	Observed on one response only; other paths may differ.
Missing Content-Security-Policy	Medium	Weaker defence against injected scripts and content.	Define a restrictive policy limiting where scripts, frames and other resources may load from.	A header proves nothing about whether the site contains vulnerable code.
Missing frame protection	Low to Medium	The site may be embedded in another page to trick users into clicking (clickjacking).	Send X-Frame-Options or a frame-ancestors directive.	Only the response headers were inspected.
Missing X-Content-Type-Options	Low	Browsers may guess content types, which can enable some attacks.	Send the header with the value nosniff.	Informational on its own.
Missing Referrer-Policy	Low	Full page addresses may leak to other sites.	Set a restrictive referrer policy.	Informational on its own.
Missing Permissions-Policy	Low	Browser features are not explicitly limited for embedded content.	Set a policy that disables unneeded features.	Informational on its own.
HTTP does not redirect to HTTPS	Medium to High	Visitors may stay on an unencrypted connection.	Redirect all HTTP requests to HTTPS and enable HSTS.	Tested from a single location.

Finding	Severity	Potential impact	Recommended remediation	Limitation
Server version disclosed	Info	Helps attackers choose known weaknesses for that version.	Hide detailed version strings.	Not proof that the version is vulnerable.
Certificate expired or not yet valid	High	Browsers warn users; identity cannot be relied on.	Renew or fix the certificate dates.	Clock differences can affect edge cases.
Hostname not covered by certificate	High	The certificate does not vouch for this site name.	Issue a certificate that lists the correct names.	Based on the name list in the presented certificate.
Untrusted or incomplete chain	High	Browsers may reject the connection.	Install the full chain from a trusted authority.	Trust stores differ slightly between systems.
Weak key or signature	Medium	Easier to attack than modern settings.	Use RSA of at least 2048 bits or a 256-bit elliptic curve, and SHA-256 or stronger.	Does not show that any key was actually broken.
Chain includes a listed distrusted CA	High	Certificate trust may be compromised, so the certificate may not reliably prove the site's identity.	Replace the certificate with one from a currently trusted authority.	This is a match to a published list. The tool cannot verify any internal key-generation weakness.
Brand impersonation in domain	High	Users may enter credentials on a site pretending to be a known organisation.	Do not enter details; report the site to the impersonated organisation.	Lexical evidence suggests impersonation but is not proof of fraud.

Appendix E - Message phrase categories (starter examples)

Signal	Example phrases to match (whole phrases, case-insensitive)
MSG-01 urgency	within 2 hours, within 24 hours, immediately, act now, last chance, expires today, urgent, final notice
MSG-02 threat	will be blocked, will be suspended, account closed, legal action, penalty, deactivated
MSG-03 sensitive request	share your otp, enter your pin, update your pan, complete kyc, verify your aadhaar, card number, cvv, login to verify
Negation guard for MSG-03	do not share, never share, don't share, we will never ask
MSG-04 payment request	pay a processing fee, send money, collect request, transfer, gift card, bitcoin, usdt, registration fee
MSG-05 too good to be true	you have won, claim your prize, cashback, refund pending, guaranteed returns, double your money, selected for the job
MSG-07 remote access	anydesk, teamviewer, quicksupport, install this app, screen share
MSG-08 obfuscated link	hxxp, dots in brackets, spaces inside links, shortener hosts from Appendix C

Appendix F - Exact user-interface copy

Where	Exact text
Every report (disclaimer)	No automated tool can guarantee a website is completely safe. This assessment reflects only the checks that were available at the time of analysis.
Certificate card (HTTPS note)	A valid HTTPS certificate only means the connection is encrypted and the certificate matches the domain. It does not mean the website is trustworthy.
Threat-feed limitation (in Limitations)	Absence of a match in these sources does not prove a site is safe.
Web3 panel (privacy)	Only a cryptographic hash, timestamp, status and reporter address are stored on-chain. Raw URLs, messages and personal data stay in our private database.
Web3 panel (truth)	A blockchain record proves a report has not changed since it was recorded. It does not prove the report is true.

Where	Exact text
AI explanation note	This explanation was generated by an AI model using only the evidence listed in this report. It can make mistakes.
Template explanation note	This explanation was generated from a fixed template using the evidence listed in this report because the AI service was unavailable.
Message box warning	Never paste passwords, OTPs, private keys or seed phrases.
Redaction notice	Sensitive-looking details were hidden before analysis.
Cached sample badge	Cached sample result
Simulation badge	SIMULATED FIXTURE - not a live finding.
SSRF block message	This address cannot be analysed because it points to a private or internal network. The SSRF guard blocked it for safety.
Language-based High Risk note	This assessment is based on the wording of the message only. It is not verified evidence of fraud.
Low Risk meaning line	No significant malicious indicators identified within the available checks.
Distrusted-chain wording	The certificate chain presented by this site includes a certificate authority certificate that appears on a published distrust list (list name, source link and date shown). This means the certificate's trust may be compromised. This tool did not and cannot independently verify any internal key-generation weakness.

Forbidden words as a verdict anywhere in output: safe, 100% safe, secure, guaranteed, completely safe, no risk. (The words may appear inside the quoted disclaimers above because they negate the claim.)

Appendix G - Glossary

Term	Meaning
SSRF	Server-side request forgery: tricking a server into contacting internal or private addresses.
TLS / HTTPS	The encryption and identity system behind secure web connections.
SAN	Subject Alternative Name: the list of names a certificate is valid for.
CA	Certificate authority: an organisation that issues certificates.
Intermediate CA	A certificate authority certificate that sits between a root and a website certificate.
CT (Certificate Transparency)	Public logs of issued certificates that help spot unexpected issuance.
RDAP	A modern lookup service for domain registration data.
Typosquatting	Registering a name that differs slightly from a real brand.
Homograph / punycode	Using look-alike characters from other alphabets; punycode is how such names are stored.
SHA-256	A hash function that turns any data into a 64-character fingerprint; any change gives a different fingerprint.
Testnet	A blockchain network for testing where tokens have no value.
Relayer	The backend wallet that signs and sends the commit transactions.
Canonical serialisation	Writing data in one fixed, repeatable text form so its hash is always the same.

Appendix H - Definition of done

- All 16 hard constraints have an automated check or a visible UI element.
- Phases 0 to 10 acceptance checks pass.
- Six demo cases run live and from cached samples; observed results recorded.
- Contract deployed on a testnet (or local network documented); one saved report with a transaction link; MATCH, MISMATCH and MATCH demonstrated.
- Frontend and backend deployed; the demo script passes on production URLs.

- README, architecture description, sources, security methodology and testing report complete; no secrets in the repository.
- Backup demo video recorded.
- Submitted on the HackMatrix dashboard at least one hour before 8:00 PM IST on 10 November 2026.